Domain and Range

My Definition

Facts & Characteristics

Domain

Examples

Non-examples

My Definition

Facts & Characteristics

Range

Examples

Non-examples

Contracts

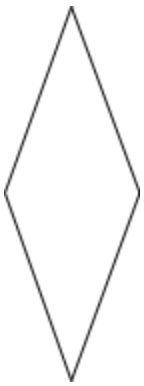
Contracts tell us how to use a function. e.g. `ellipse :: Number, Number, String, String -> Image` tells us that the name of the function is `ellipse`, and that it takes four inputs (two Numbers and two Strings). From the contract, we know `(ellipse 100 50 "outline" "teal")` will evaluate to an Image.

Name		Domain		Range
`; triangle`	`::`	`Number, String, Sting`	`->`	`Image`
(triangle 80 "solid" "green")				
; star	::	Number, String, String	->	Image
(star 50 "solid" "teal")				
; circle	::	Number, String, String	->	Image
(circle 45 "outline" "darkgreen")				
; square	::	Number, String, String	->	Image
(square 70 "outline" "lightblue")				
; rectangle	::	Number, Number, String, String	->	Image
(rectangle 20 80 "solid" "gold")				
; text	::	String, Number, String	->	Image
(text "I'm thankful for…" 50 "brown")				
; ellipse	::	Number, Number, String, String	->	Image
(ellipse 70 30 "outline" "red")				
; regular-polygon	::	Number, Number, String, String	->	Image
(regular-polygon 40 8 "solid" "red")				
; rhombus	::	Number, Number, String, String	->	Image
(rhombus 20 50 "solid" "turquoise")				
; right-triangle	::	Number, Number, String, String	->	Image

Using Contracts (continued)

Use the contracts to write expressions to generate images similar to those pictured.

`rhombus :: Number, Number, String, String -> Image`

	`(rhombus 100 140 "solid" "black")`
	`(rhombus 100 40 "outline" "black")`
Write an expression for a square (rotated) using `rhombus`!	`(rhombus 100 90 "solid" "black")`
What variable changes with the first Number?	*The length of each side*
What variable changes with the second Number?	*The top and bottom angle of the rhombus*

Triangle Contracts (SAS & ASA)

Type each expression (left) below into the editor and match it to the image it creates (right).

Expression			Image
`(triangle-sas 120 45 70 "solid" "black")`	1	A	
`(triangle-sas 120 90 70 "solid" "black")`	2	B	
`(triangle-sas 120 135 70 "solid" "black")`	3	C	
`(triangle-sas 70 135 120 "solid" "black")`	4	D	

Think about how you would describe each of the arguments that `triangle-sas` takes in to someone who'd never used the function before and annotate the contract below using descriptive variable names.

```
triangle-sas :: (
    ____________________side1____________________ :: Number,
    ____________________angle____________________ :: Number,
    ____________________side2____________________ :: Number,
    ____________________style____________________ :: String,
    ____________________color____________________ :: String
) -> Image
```

Add examples of each of the triangle functions we've explored to your contracts page.

If you have time, experiment with the `triangle-asa` function.

```
triangle-asa :: (
    left-angle :: Number,
    left-side :: Number,
    bottom-angle :: Number,
    style :: String
    color :: String
) -> Image
```

Star Polygon

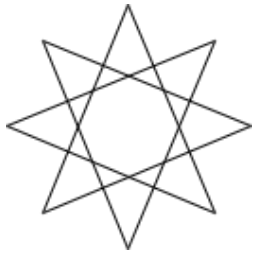
```
star-polygon :: (

    side-length :: Number,
    points-on-polygon :: Number,
    polygon-points-to-skip-between-star-points :: Number,
    shading-style  :: String,
    color :: String

) -> Image
```

Using the detailed contract above, write expressions to create each image below.

Then make two more star polygons of your choosing. Sketch them and write expressions to generate them.

	`(star-polygon 50 8 3 "outline" "black")`
	`(star-polygon 50 8 2 "solid" "black")`

Mapping Examples with Circles of Evaluation

Contract: _____

Purpose Statement: _____

If I type…	→	It should map to…
EXAMPLE #1: Circle of Evaluation Code:	→	Circle of Evaluation: Code:
EXAMPLE #2: Circle of Evaluation Code:	→	Circle of Evaluation: Code:

Do The Examples Have the Same Contracts?

For each pair of Examples below, decide whether the two examples have the same contract. If they do, fill in the contract in the space provided. If not, write a few words explaining how you know their contracts aren't the same.

1) no match (the range types are different)

```
(EXAMPLE (mystery 30) (* 30 50))
(EXAMPLE (mystery 10)
  (text "Welcome!" 10 "darkgreen"))
```

2) ; mystery :: Number, Number -> Number

```
(EXAMPLE (mystery 30 40) (- 40 (* 2 30)))
(EXAMPLE (mystery 10 15) (- 15 (* 2 10)))
```

3) no match (the domain types are different)

```
(EXAMPLE (mystery "New York")
  (text "New York" 20 "red"))
(EXAMPLE (mystery 20)
  (text "New York" 20 "red"))
```

4) no match (the domain types are in different orders)

```
(EXAMPLE (mystery "green" 32)
  (circle 32 "outline" "green"))
(EXAMPLE (mystery 18 "green")
  (circle 18 "outline" "green"))
```

5) no match (the domains have different numbers of arguments)

```
(EXAMPLE (mystery 6 9 10) (/ 6 (+ 9 10)))
(EXAMPLE (mystery 3 7) (/ 3 (+ 7 10)))
```

6) ; mystery :: String, String -> Image

```
(EXAMPLE (mystery "red" "blue")
  (text "blue" 25 "red"))
(EXAMPLE (mystery "purple" "Go Team!")
  (text "Go Team!" 25 "purple"))
```

Do The Examples Have the Same Contracts? (2)

For each pair of Examples below, decide whether the two examples have the same contract. If they do, fill in the contract in the space provided. If not, write a few words explaining how you know their contracts aren't the same.

1) `; mystery :: Image -> Image`

```
(EXAMPLE (mystery (triangle 70 "solid" "green"))
  (triangle 140 "solid" "green"))
(EXAMPLE (mystery (circle 100 "solid" "blue"))
  (circle 200 "solid" "blue"))
```

2) no match (the domains have different numbers of arguments)

```
(EXAMPLE (mystery "red")
  (triangle 140 "solid" "red"))
(EXAMPLE (mystery "blue" "circle")
  (circle 140 "solid" "blue"))
```

3) no match (the domains have different numbers of arguments)

```
(EXAMPLE (mystery "" 4 5) ( 4 5))
(EXAMPLE (mystery "sqrt" 25) (sqrt 25))
```

4) `; mystery :: String, Number -> Number`

```
(EXAMPLE (mystery "circle" 4) (* pi (sqr 4)))
(EXAMPLE (mystery "square" 5) (sqr 5))
```

5) no match (the range types are different)

```
(EXAMPLE (mystery "dog") 3)
(EXAMPLE (mystery "cat") "kitten")
```

6) `; mystery :: String -> Number`

```
(EXAMPLE (mystery "dog") 3)
(EXAMPLE (mystery "kitten") 6)
```

Matching Contracts and Examples

Match each Example on the left with its Contract on the right. NOTE: Multiple examples may match to the same contract!

Contract			Examples
`(EXAMPLE (match (circle 10 "solid" "green")) (rotate 37 (circle 10 "solid" "green")))`	**1**-C	**A**	`; match :: Number, Image -> Image`
`(EXAMPLE (match (triangle 20 "solid" "blue") 3) (scale 3 (triangle 20 "solid" "blue")))`	**2**-B		
`(EXAMPLE (match (circle 20 "outline" "gold")) (rotate 37 (circle 20 "outline" "gold")))`	**3**-C	**B**	`; match :: Image, Number -> Image`
`(EXAMPLE (match 30 "red") (+ 30 (string-length "red")))`	**4**-D		
`(EXAMPLE (match (circle 10 "solid" "orange") 22) (scale 22 (circle 10 "solid" "orange")))`	**5**-B		
`(EXAMPLE (match 10 "blue") (+ 10 (string-length "blue")))`	**6**-D	**C**	`; match :: Image -> Image`
`(EXAMPLE (match 5 (star 20 "solid" "red")) (rotate (- 90 5) (star 20 "solid" "red")))`	**7**-A		
`(EXAMPLE (match (abs -4) "45") 4)`	**8**-D	**D**	`; match :: Number, String -> Number`

Matching Contracts and Examples 2

Match each Example on the left with its Contract on the right. NOTE: Multiple examples may match to the same contract!

Contract		Examples	
`(EXAMPLE (match 1.5) "greater than 1")`	1-A		
`(EXAMPLE (match 24)` `  (star (* 24 2) "outline" "purple"))`	2-B		
`(EXAMPLE (match (string-length "tabletop"))` `  "8")`	3-A	A	`; match :: Number -> String`
`(EXAMPLE (match (star 20 "outline" "red") 3)` `  (* 3 (image-height (star 20 "outline" "red"))))`	4-E	B	`; match :: Number -> Image`

`(EXAMPLE (match (circle 10 "solid" "silver") 16)`
`  (* 16 (image-height (circle 10 "solid"`
`"silver"))))`
	5-E	C	`; match :: Number, Number -> Number`

`(EXAMPLE (match "triangle" "blue")` `  (triangle 40 "outline" "blue"))`	6-D	D	`; match :: String, String -> Image`
`(EXAMPLE (match 30)` `  (star (* 30 2) "outline" "purple"))`	7-B	E	`; match :: Images, Number -> Number`

`(EXAMPLE (match (string-length "coffee") (string-`
`length "tea"))`
`  (+ 6 3))`
	8-C		